



Attention Is All You Need 논문 정리

1. 문제의식과 제안

1.1 기존 구조의 한계

- **RNN/LSTM/GRU**: 위치 t 의 hidden state가 $t - 1$ 에 의존하므로 한 문장 안에서 병렬 계산하기 어렵다. 문장이 길수록 순차 연산 병목이 커진다.
- **CNN 기반 sequence model**: 모든 위치는 병렬 처리할 수 있지만, 멀리 떨어진 두 토큰을 연결하려면 여러 층을 통과해야 한다. 즉 장거리 의존성의 path length가 길어진다.
- 기존 attention은 대개 RNN 또는 CNN의 보조 모듈이었다.

1.2 논문의 제안

- encoder와 decoder를 유지하되, 내부의 핵심 연산을 **multi-head attention + position-wise feed-forward network**로 구성한다.
- recurrence와 convolution을 모두 제거한다.
- 토큰 순서는 positional encoding으로 따로 주입한다.

2. 전체 구조

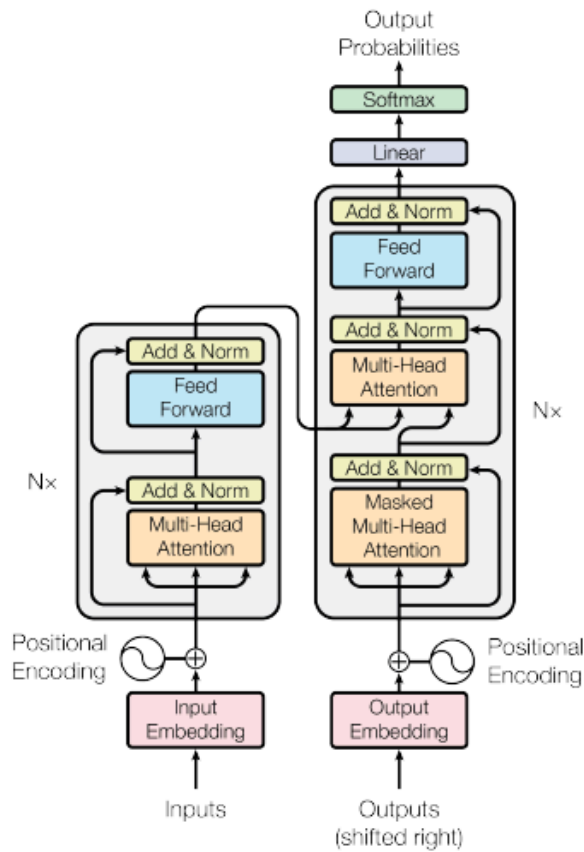


Figure 1: The Transformer - model architecture.

구분	Encoder	Decoder
층 수	$N = 6$	$N = 6$
주요 sub-layer	Multi-head self-attention → FFN	Masked self-attention → encoder-decoder attention → FFN
정보 접근 범위	입력의 모든 위치	이미 생성된 출력 위치 + encoder 전체 출력
공통 구조	각 sub-layer에 residual connection과 layer normalization	각 sub-layer에 residual connection과 layer normalization

원 논문의 각 sub-layer 출력은 다음과 같다.

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

이는 **post-LN** 구조다. 즉 residual addition 뒤에 LayerNorm을 적용한다. 이후의 많은 Transformer 변형은 학습 안정성을 위해 pre-LN을 사용하므로 구현 코드를 볼 때 구분해야 한다.

3. Attention의 핵심

3.1 Query, Key, Value의 직관

- **Query (Q)**: 현재 토큰이 "어떤 정보를 찾고 있는가?"
- **Key (K)**: 각 토큰이 "어떤 정보를 가지고 있다고 표시하는가?"
- **Value (V)**: 실제로 가중합해 가져올 정보

현재 query와 각 key의 유사도로 attention weight를 만들고, 그 weight로 value를 가중평균한다.

3.2 Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (1)$$

계산 순서

1. $QK^\top$: 각 query와 모든 key 사이의 유사도 점수를 계산한다.
2. $\sqrt{d_k}$ 로 나누기: 점수의 크기를 안정화한다.
3. softmax: 각 행을 합이 1인 attention weight로 바꾼다.
4. V 와 곱하기: 관련성이 높은 value를 더 많이 반영한 새 표현을 만든다.

3.3 왜 $\sqrt{d_k}$ 로 나누는가?

q_i 와 k_i 가 평균 0, 분산 1이고 서로 독립이라고 보면,

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i, \quad \text{Var}(q \cdot k) = d_k.$$

따라서 내적의 표준편차는 $\sqrt{d_k}$ 수준으로 커진다. 이 값을 그대로 softmax에 넣으면 확률이 한쪽으로 치나치게 몰려 gradient가 매우 작아질 수 있다. $\sqrt{d_k}$ 로 나누면 score의 스케일을 대략 일정하게 유지할 수 있다.

3.4 텐서 차원으로 보기

Encoder self-attention에서 입력을 $X \in \mathbb{R}^{n \times d_{\text{model}}}$ 라고 하면, head 하나에서

- $Q = XW^Q \in \mathbb{R}^{n \times d_k}$
- $K = XW^K \in \mathbb{R}^{n \times d_k}$
- $V = XW^V \in \mathbb{R}^{n \times d_v}$
- attention score $QK^\top \in \mathbb{R}^{n \times n}$

- head 출력 $\in \mathbb{R}^{n \times d_v}$

원 논문의 base model은 $d_{model} = 512, h = 8, d_k = d_v = 64$ 이다.

3.5 Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O,$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V).$$

하나의 큰 attention만 쓰지 않고 여러 저차원 head를 병렬로 사용하면, 서로 다른 representation subspace와 서로 다른 위치 관계에 동시에 주의를 기울일 수 있다. 예를 들어 어떤 head는 문법적 대응, 다른 head는 장거리 지시 관계를 포착할 수 있다.

4. Transformer에서 쓰이는 세 가지 attention

종류	Q 출처	K, V 출처	역할
Encoder self-attention	encoder 이전 층	encoder 이전 층	입력 토큰끼리 전역 관계 학습
Masked decoder self-attention	decoder 이전 층	decoder 이전 층	현재 위치가 미래 출력 토큰을 보지 못하게 차단
Encoder-decoder attention	decoder 이전 sub-layer	encoder 최종 출력	출력 생성 시 입력 문장의 관련 위치 참조

5. Attention 외의 구성요소

5.1 Position-wise Feed-Forward Network

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2. \quad (2)$$

- 각 위치에 **동일한 두 층 MLP**를 독립적으로 적용한다.
- 위치끼리 정보를 섞는 것은 attention이고, 각 위치의 feature를 비선형 변환하는 것은 FFN이다.
- base model: 입력/출력 $d_{model} = 512$, 내부 차원 $d_{ff} = 2048$.

5.2 Positional Encoding

recurrence와 convolution이 없으면 토큰 순서가 자동으로 들어오지 않는다. 따라서 embedding과 같은 크기의 위치 벡터를 더한다.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right).$$

- 여러 주기의 sine/cosine을 사용하여 각 위치에 고유한 패턴을 부여한다.
- 저자들은 고정 offset k 에 대해 PE_{pos+k} 가 PE_{pos} 의 선형함수로 표현될 수 있어 상대 위치를 학습하기 쉬운 것이라고 가정했다.
- learned positional embedding과 성능이 거의 같았지만, 학습 때보다 긴 sequence로의 외삽 가능성을 기대해 sinusoidal encoding을 선택했다.

5.3 Embedding과 출력 Softmax

- 입력·출력 토큰을 d_{model} 차원 learned embedding으로 바꾼다.
- 입력 embedding matrix, 출력 embedding matrix, pre-softmax linear transformation의 weight를 공유한다.
- embedding에 $\sqrt{d_{model}}$ 을 곱해 positional encoding과 합산한다.

6. 왜 Self-Attention인가?

논문은 세 가지 기준으로 self-attention, recurrence, convolution을 비교한다.

Layer	층당 복잡도	최소 순차 연산	최대 path length
Self-attention	$O(n^2d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$
Convolutional	$O(knd^2)$	$O(1)$	$O(\log_k n)$
Restricted self-attention	$O(rnd)$	$O(1)$	$O(n/r)$

여기서 n 은 sequence length, d 는 representation dimension, k 는 convolution kernel size, r 은 local attention neighborhood 크기다.

해석

- self-attention은 모든 위치를 한 층에서 직접 연결하므로 장거리 의존성의 path length가 1이다.
- 문장처럼 대체로 $n < d$ 인 경우 $n^2d < nd^2$ 이므로 RNN보다 계산량도 유리할 수 있다.
- 반대로 n 이 매우 긴 경우 n^2 항이 병목이 된다. 논문도 이를 인식하고 restricted/local attention을 향후 연구로 제안한다.

7. 학습 설정

7.1 데이터와 batching

- WMT 2014 English-German: 약 **450만 문장쌍**, shared BPE vocabulary 약 37,000.
- WMT 2014 English-French: 약 **3,600만 문장쌍**, word-piece vocabulary 32,000.
- 비슷한 길이의 문장을 묶고, batch당 source 약 25,000 tokens + target 약 25,000 tokens.

7.2 Base와 Big

설정	Base	Big
N	6	6
d_{model}	512	1024
d_{ff}	2048	4096
heads	8	16
dropout	0.1	0.3 (영-불 big은 0.1)
train steps	100K	300K
parameters	65M	213M
학습 시간	12시간	3.5일

하드웨어는 한 대의 머신에 NVIDIA P100 GPU 8개였다.

7.3 Optimizer와 learning-rate schedule

Adam: $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$.

$$lrate = d_{model}^{-0.5} \min(step^{-0.5}, step \cdot warmup_steps^{-1.5}), \quad warmup_steps = 4000$$

초기 4,000 step은 learning rate를 선형 증가시키고, 이후에는 step의 역제곱근에 비례해 감소시킨다.

7.4 Regularization과 추론

- residual dropout: sub-layer 출력과 embedding+positional encoding 합에 적용.
- label smoothing: $\epsilon_{ls} = 0.1$. perplexity는 나빠질 수 있지만 accuracy와 BLEU는 좋아졌다.
- 번역 추론: beam size 4, length penalty $\alpha = 0.6$; base는 마지막 5개, big은 마지막 20개 checkpoint 평균.

8. 실험 결과

8.1 기계번역

모델	EN-DE BLEU	EN-FR BLEU	추정 학습비용
Transformer base	27.3	38.1	3.3×10^{18} FLOPs
Transformer big	28.4	41.8	2.3×10^{19} FLOPs

- 영-독에서 기존 최고 결과(ensemble 포함)를 2 BLEU 이상 개선했다.
- 영-불에서는 당시 single-model state of the art를 훨씬 낮은 학습비용으로 달성했다.

8.2 Ablation에서 확인된 점

- single head: BLEU 24.9로 base의 25.8보다 0.9 낮았다.
- head 16개는 25.8로 유지했지만 32개는 25.4로 하락했다.
- key dimension d_k 을 줄이면 성능이 나빠졌다. query-key compatibility 계산에 충분한 차원이 필요함을 시사한다.
- model/FFN 차원을 키우면 대체로 성능이 좋아졌다.
- dropout은 과적합 방지에 중요했다.
- sinusoidal positional encoding(25.8)과 learned positional embedding(25.7)은 거의 같았다.

8.3 English constituency parsing

- WSJ 40K 문장만 사용: F1 **91.3**.
- semi-supervised 약 17M 문장 사용: F1 **92.7**.
- task-specific tuning이 많지 않았는데도 강한 성능을 보여, 번역 이외의 structured output task로 일반화될 가능성을 보였다.